Submission Worksheet

Submission Data

Course: IT114-007-F2025
Assignment: IT114 Milestone 1
Student: Ryan C. (rc728)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 11/5/2025 1:41:03 AM
Updated: 11/5/2025 11:38:36 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-1/grading/rc728
View Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-1/view/rc728

Instructions

- Overview Link: https://youtu.be/9dZPFwi76ak

1. Refer to Milestone1 of any of these docs:
    2. Rock Paper Scissors
    3. Basic Battleship
    4. Hangman / Word guess
    5. Trivia
    6. Go Fish
    7. Pictionary / Drawing
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Copy `Part5` and rename the copy as `Project` (this new folder should be in the root of your repo)
6. Organize the files into their respective packages `Client`, `Common`, `Server`, `Exceptions`
    1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson
7. Fill out the below worksheet
    1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.
8. Once finished, click "Submit and Export"
9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `main`
10. Upload the same PDF to Canvas
11. Sync Local

1. git checkout main
2. git pull origin main

Section #1: (1 pt.) Feature: Server Can Be Started Via Command Line And Listen To Connections

Progress: 100%

Task #1 (1 pt.) - Evidence

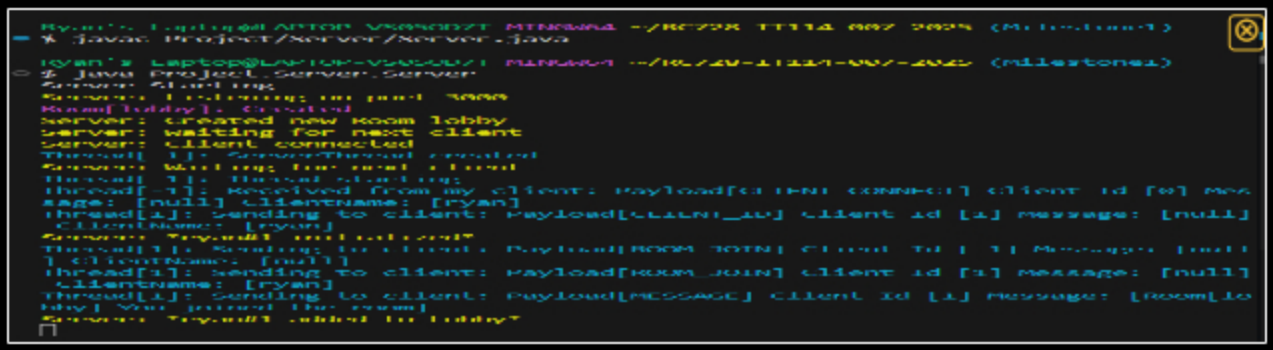
Progress: 100%

Part 1:

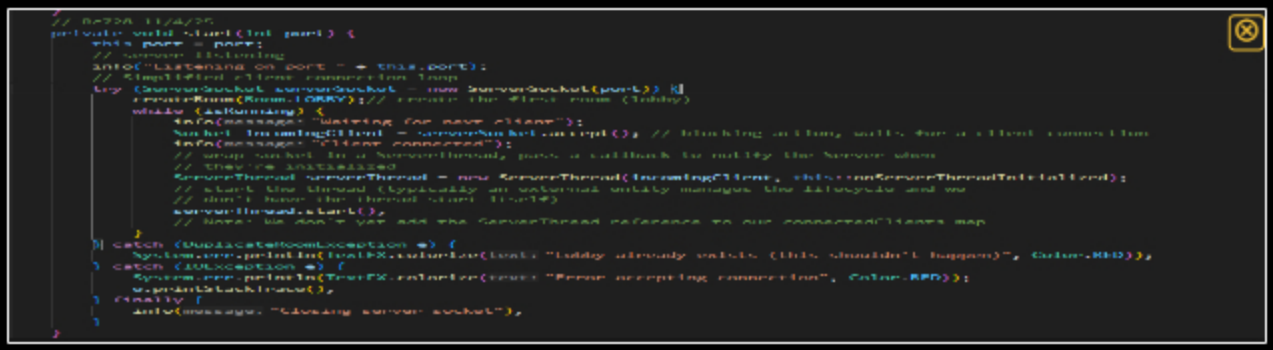
Progress: 100%

Details:

- Show the terminal output of the server started and listening
- Show the relevant snippet of the code that waits for incoming connections



this shows the terminal output of the server starting up and listening for the connection as well as the client connecting



this shows the code that awaits for the incoming connections

Saved: 11/5/2025 2:01:26 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side waits for and accepts/handles connections

Your Response:

The server-side waits for the connection by when the server is activated, it will activate the while loop watiing for a response which would first generate a "waiting for next client" until a client connects which is when the accept method returns a new socket object representing that connection. Then a new ServerThread is created by the server to handle any communication with the client so the server can keep the loop going waiting for other clients to join.

Saved: 11/5/2025 2:01:26 AM

Section #2: (1 pt.) Feature: Server Should Be Able To Allow More Than One Client To Be Connected At Once

Progress: 100%

Task #1 (1 pt.) - Evidence

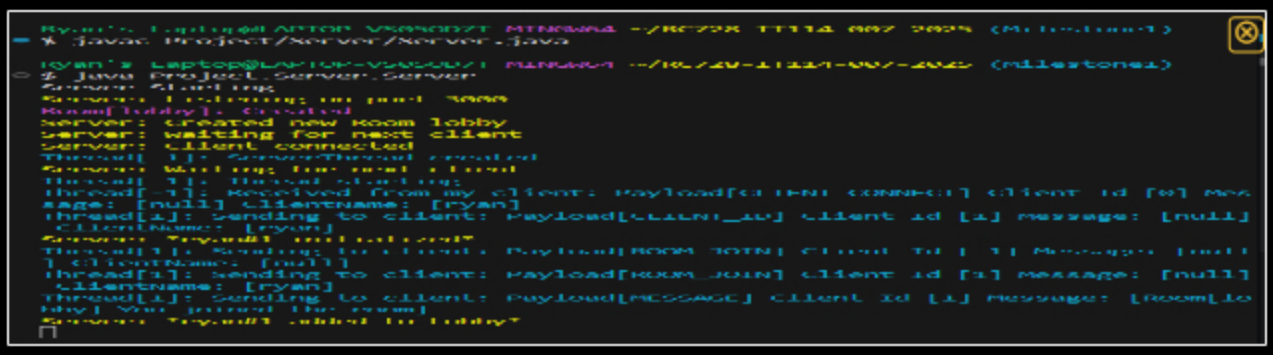
Progress: 100%

Part 1:

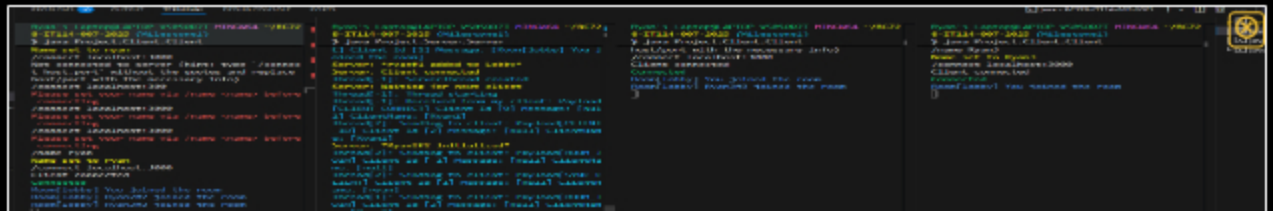
Progress: 100%

Details:

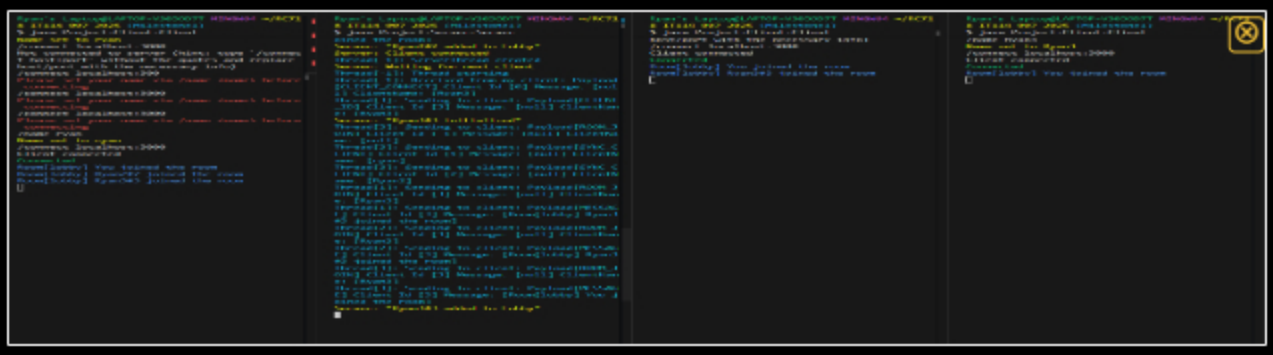
- Show the terminal output of the server receiving multiple connections
- Show at least 3 Clients connected (best to use the split terminal feature)
- Show the relevant snippets of code that handle logic for multiple connections



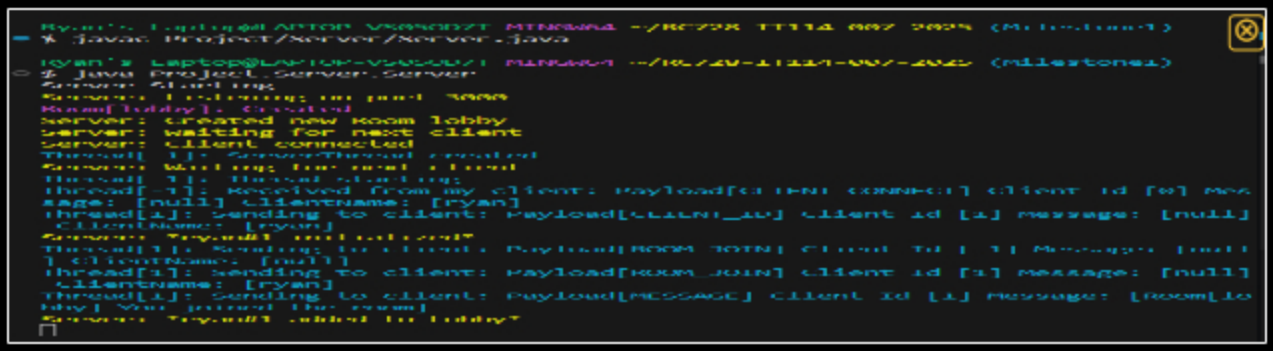
this shows the first client joining the server which is just the name of ryan



this shows the second client joining the server which has the name of ryan2



This shows the third client joining the server which ahs the name of ryan3



This also shows the code on how the server can handle multiple connections by giving each client a serverthread

Saved: 11/5/2025 2:23:38 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles multiple connected clients

Your Response:

The server handles multiple connections by giving each client thier own serverthread with each serverthread running independently allowing for the client to communicate certain things just for that serverthread while the server is allowed to continue to listen for more connections.

Saved: 11/5/2025 2:23:38 AM

Section #3: (2 pts.) Feature: Server Will Implement The Concept Of Rooms (With The

Implement The Concept Of Rooms (With The Default Being "l obby")

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of rooms being created, joined, and removed (server-side)
- Show the relevant snippets of code that handle room management (create, join, leave, remove) (server-side)

```
Room[MyRoom]: Created
Server: Created new Room MyRoom
Server: Removing client from previous Room lobby
Thread[1]: Sending to client: Payload[ROOM_LEAVE] Client Id [1] Message: [null
] ClientName: [Ryan1]
Thread[1]: Sending to client: Payload[MESSAGE] Client Id [1] Message: [Room[lo
bby] You left the room]
Thread[2]: Sending to client: Payload[ROOM_LEAVE] Client Id [1] Message: [null
] ClientName: [Ryan1]
Thread[2]: Sending to client: Payload[MESSAGE] Client Id [1] Message: [Room[lo
bby] Ryan1#1 left the room]
Thread[1]: Sending to client: Payload[ROOM_JOIN] Client Id [-1] Message: [null
] ClientName: [null]
Thread[1]: Sending to client: Payload[ROOM_JOIN] Client Id [1] Message: [null]
 ClientName: [Ryan1]
Thread[1]: Sending to client: Payload[MESSAGE] Client Id [1] Message: [Room[My
Room] You joined the room]
Thread[2]: Received from my client: Payload[ROOM_JOIN] Client Id [0] Message:
[MyRoom]
```

This shows the server-side of the room being created and the client leaving the lobby to join the room

```
Server: Removing client from previous Room MyRoom
Thread[1]: Sending to client: Payload[ROOM_LEAVE] Client Id [1] Message: [null
] ClientName: [Ryan1]
Thread[1]: Sending to client: Payload[MESSAGE] Client Id [1] Message: [Room[My
Room] You left the room]
Thread[2]: Sending to client: Payload[ROOM_LEAVE] Client Id [1] Message: [null
] ClientName: [Ryan1]
Thread[2]: Sending to client: Payload[MESSAGE] Client Id [1] Message: [Room[My
Room] Ryan1#1 left the room]
Thread[1]: Sending to client: Payload[ROOM_JOIN] Client Id [-1] Message: [null
] ClientName: [null]
Thread[1]: Sending to client: Payload[ROOM_JOIN] Client Id [1] Message: [null]
 ClientName: [Ryan1]
Thread[1]: Sending to client: Payload[MESSAGE] Client Id [1] Message: [Room[lo
bby] You joined the room]
Thread[2]: Received from my client: Payload[ROOM_LEAVE] Client Id [0] Message:
 [MyRoom]
Server: Removing client from previous Room MyRoom
Thread[2]: Sending to client: Payload[ROOM_LEAVE] Client Id [2] Message: [null
] ClientName: [Ryan2]
Thread[2]: Sending to client: Payload[MESSAGE] Client Id [2] Message: [Room[My
Room] You left the room]
Server: Removed room MyRoom
Room[MyRoom]: closed
```

This shows the servers-side of all the clients leaving the room and the room being removed

```java
// start handle methods
//rc728 11/5/25
public void handleCreateRoom(ServerThread sender, String roomName) {
    try {
        Server.INSTANCE.createRoom(roomName);
        Server.INSTANCE.joinRoom(roomName, sender);
    } catch (RoomNotFoundException e) {
        info(message: "Room wasn't found (this shouldn't happen)");
        e.printStackTrace();
    } catch (DuplicateRoomException e) {
        sender.sendMessage(Constants.DEFAULT_CLIENT_ID, String.format(format: "Room %s already exists", roomName));
    }
}
```

This shows the code that allows for the creation of the room

```java
//Rc728 11/5/25
```

```java
public void handleJoinRoom(ServerThread sender, String roomName) {
    try {
        Server.INSTANCE.joinRoom(roomName, sender);
    } catch (RoomNotFoundException e) {
        sender.sendMessage(Constants.DEFAULT_CLIENT_ID, String.format(format: "Room %s doesn't exist", roomName));
    }
}
```

this shows the code that allows for the joining of the room

```java
protected synchronized void removeClient(ServerThread client) {
    if (!isRunning) { // block action if Room isn't running
        return;
    }
    if (!clientsInRoom.containsKey(client.getClientId())) {
        info(message: "Attempting to remove a client that doesn't exist in the room");
        return;
    }
    ServerThread removedClient = clientsInRoom.get(client.getClientId());
    if (removedClient != null) {
        // notify clients of someone joining
        joinStatusRelay(removedClient, didJoin: false);
        clientsInRoom.remove(client.getClientId());
        autoCleanup();
    }
}
```

this shows the code that allows for the client to be removed if they leave

```java
private void autoCleanup() {
    if (!Room.LOBBY.equalsIgnoreCase(name) && clientsInRoom.isEmpty()) {
        close();
    }
}

@Override
public void close() {
    // attempt to gracefully close and migrate clients
    if (!clientsInRoom.isEmpty()) {
        relay(sender: null, message: "Room is shutting down, migrating to lobby");
        info(String.format(format: "Migrating %s clients", clientsInRoom.size()));
        clientsInRoom.values().removeIf(client -> {
            try {
                Server.INSTANCE.joinRoom(Room.LOBBY, client);
            } catch (RoomNotFoundException e) {
                e.printStackTrace();
                // FIXME if this should never happen
            }
            return true;
        });
    }
    Server.INSTANCE.removeRoom(this);
    isRunning = false;
    clientsInRoom.clear();
    info(String.format(format: "Closed"));
}
```

This shows the code for the room to be removed if there isn't any clients in the room

Saved: 11/5/2025 5:58:45 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles room creation, joining/leaving, and removal

Your Response:

the server-side handles the room handling throught giving each client a serverthread to handle each seperately to allow for each client to do their own seperate commands. The rooms are handled through the server and room class however. When the "/create roomName" command is ran in the terminal, the server will execute the handleCreateRoom() method which would create a new room object and then it will be added to the server's list of all the active rooms. The client that made the room is then automatically removed from the lobby and added to the newly created room. When the client executes the "/join roomName" command, the server first checks if the room exists, if it does, the server added the client's serverthread to the room's list of all connected

users. Then a message is broadcasted to everyone in the room letting them know that a new user has joined the room. If the client runs the "/leave" command, the server will remove the client from the room and notifies all users that are still in the room that the user has left the room. If the room is empty and has no more users in the room, the server will automatically closes and removes the room from the active rooms list.

Saved: 11/5/2025 5:58:45 PM

Section #4: (1 pt.) Feature: Client Can Be Started Via The Command Line

Progress: 100%

Task #1 (1 pt.) - Evidence

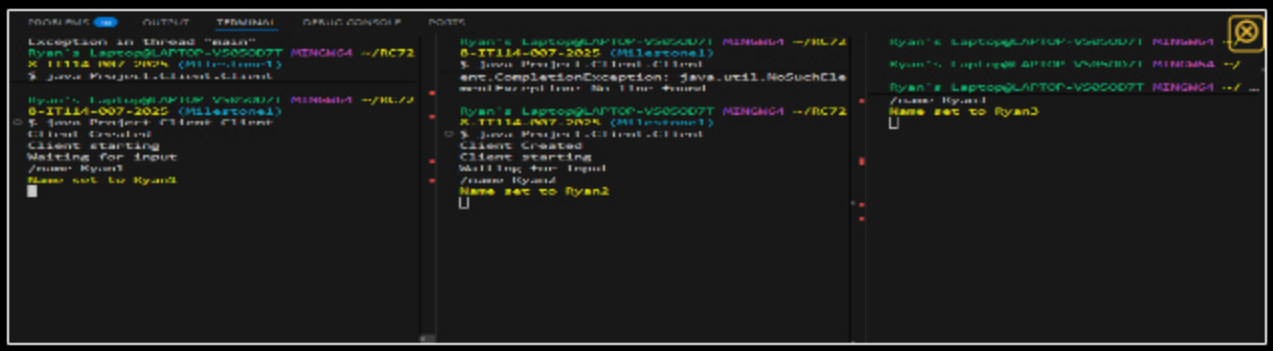
Progress: 100%

Part 1:

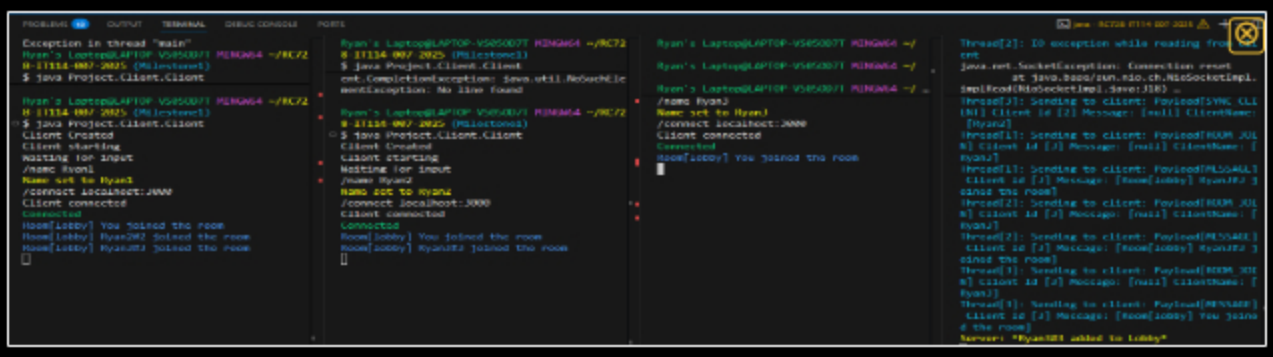
Progress: 100%

Details:

- Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)
- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected



This shows the terminal output of the /name command



This shows the terminal output of the /connect command

```
    private boolean connect(String address, int port) {
        try {
            server = new Socket(address, port);
            // channel to send to server
            out = new ObjectOutputStream(server.getOutputStream());
            // channel to listen to server
            in = new ObjectInputStream(server.getInputStream());
            System.out.println(x: "Client connected");
            // Use CompletableFuture to run listenToServer() in a separate thread
            CompletableFuture.runAsync(this::listenToServer);
        } catch (UnknownHostException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
        return isConnected();
    }
```

This shows the code for the connect command

```
//Rachel 11/5/25
private boolean processClientCommand(String text) throws IOException {
    boolean wasCommand = false;
    if (text.startsWith(Constants.COMMAND_TRIGGER)) {
        text = text.substring(beginIndex: 1); // remove the /
        // System.out.println("Processing command: " + text);
        if (!isConnected("/" + text)) {
            if (myUser.getClientName() == null || myUser.getClientName().isEmpty()) {
                System.out.println(
                    TextFX.colorize(text: "Please set your name via /name <name> before connecting", Color.RED));
                return true;
            }
            // replaces multiple spaces with a single space
            // splits on the space after connect (gives us host and port)
            // splits on : to get host as index 0 and port as index 1
            String[] parts = text.trim().replaceAll(regex: " +", replacement: " ").split(regex: " ")[1].split(regex: ":");
            connect(parts[0].trim(), Integer.parseInt(parts[1].trim()));
            sendClientName(myUser.getClientName());// sync follow-up data (handshake)
            wasCommand = true;
        } else if (text.startsWith(Command.NAME.command)) {
            text = text.replace(Command.NAME.command, replacement: "").trim();
            if (text == null || text.length() == 0) {
                System.out.println(TextFX.colorize(text: "This command requires a name as an argument", Color.RED));
                return true;
            }
            myUser.setClientName(text);// temporary until we get a response from the server
            System.out.println(TextFX.colorize(String.format(format: "Name set to %s", myUser.getClientName()),
                Color.YELLOW));
            wasCommand = true;
```

This shows the code for the name command

Saved: 11/5/2025 7:30:47 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the /name and /connect commands work and the code flow that leads to a successful connection for the client

Your Response:

when the client enter the "/name" command followed by a name, the client program will the run the processClientCommand() method in the client.java. This command allows for the /name part to be removed and leaves the name and stores it in the myUser object using the myUser.getClientName() method. Then a message is sent telling the user that name is set to the name they had given. when the "/connect" command is executed, processClientCommand() method sees the command and takes the ip address and the port from the command. It then calls the connect() method which would create a socket connection to the server which would set input and output streams for communication. It then sends a message saying that the client is connected. After the server accepts the connection,it will send a message saying the client has joined with it including the name and the clisent ID. The Client processes this response by storing the assigned clident ID and will display a green "connected" text.

Saved: 11/5/2025 7:30:47 PM

Section #5: (2 pts.) Feature: Client Can Create/j oin Rooms

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the /createroom and /joinroom
- Output should show evidence of a successful creation/join in both scenarios
- Show the relevant snippets of code that handle the client-side processes for room creation and joining

```
/createroom MyRoom
Room[lobby] You left the room
Room[MyRoom] You joined the room
```

this shows /createroom command being used and working

```
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
/createroom MyRoom
Room[lobby] You left the room
Room[MyRoom] You joined the room
Room[MyRoom] Ryan2#2 joined the room
```

```
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan3#3 joined the room
Room[lobby] Ryan1#1 left the room
/joinroom MyRoom
Room[lobby] You left the room
Room[MyRoom] You joined the room
```

this show the /joinroom command being used with it working

```
} else if (text.startsWith(Command.CREATE_ROOM.command)) {
    text = text.replace(Command.CREATE_ROOM.command, replacement: "").trim();
    if (text == null || text.length() == 0) {
        System.out.println(TextFX.colorize(text: "This command requires a room name as an argument", Color.RED));
        return true;
    }
    sendRoomAction(text, RoomAction.CREATE);
    wasCommand = true;
```

this shows the code for the /createroom command

```
        //rc728 11/5/25
    } else if (text.startsWith(Command.JOIN_ROOM.command)) {
        text = text.replace(Command.JOIN_ROOM.command, replacement: "").trim();
        if (text == null || text.length() == 0) {
            System.out.println(TextFX.colorize(text: "This command requires a room name as an argument", Color.RED));
            return true;
        }
        sendRoomAction(text, RoomAction.JOIN);
        wasCommand = true;
```

This shows the code for the /joinroom command

Saved: 11/5/2025 8:18:24 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the /createroom and /join room commands work and the related code flow for each

Your Response:

When the client uses the /createroom command, the processClientCommand() method is ran which would call the sendRoomAction() method with the RoomAction.Create. this would send a create room payload to the server which would create the room and adds the client while also notifying all the clients. Then there is a confirmation method which would show the room was made successfully. The /joinroom command works by also having the processClientCommand() method call sendRoomAction(). this will send a join room payload to the sever which would add the client to the room if it exists and messages all the clients about the user joining the room. A confirmation message will also be sent to the joined client.

Saved: 11/5/2025 8:18:24 PM

Section #6: (1 pt.) Feature: Client Can Send Messages

Progress: 100%

Task #1 (1 pt.) - Evidence

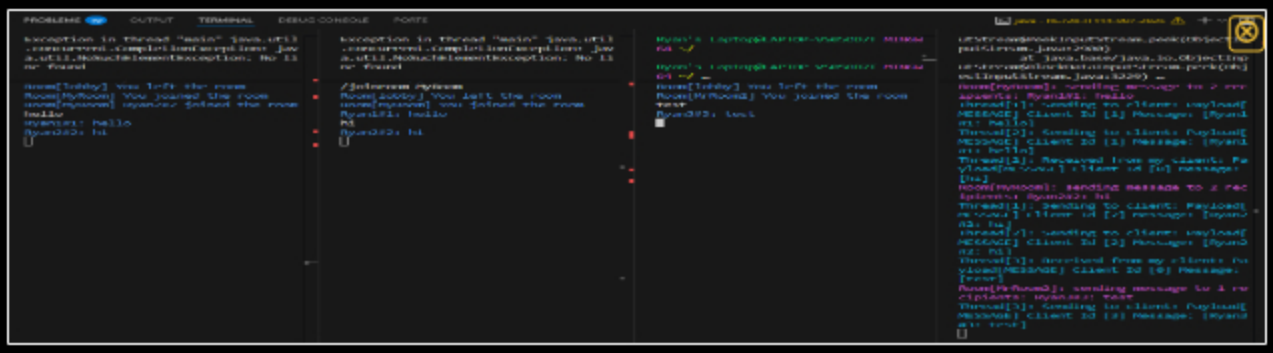
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of a few messages from each of 3 clients
- Include examples of clients grouped into other rooms
- Show the relevant snippets of code that handle the message process from client to server-side and back



this shows clients speaking with two in the same room and 1 in a different room as well as the server showing it

```java
//Rc728 11/5/25
private void sendMessage(String message) throws IOException {
    Payload payload = new Payload();
    payload.setMessage(message);
    payload.setPayloadType(PayloadType.MESSAGE);
    sendToServer(payload);
}
```

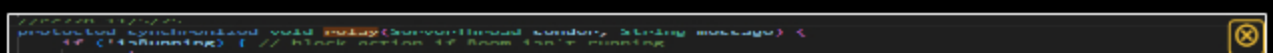
This shows the code for how clients are able to send messages

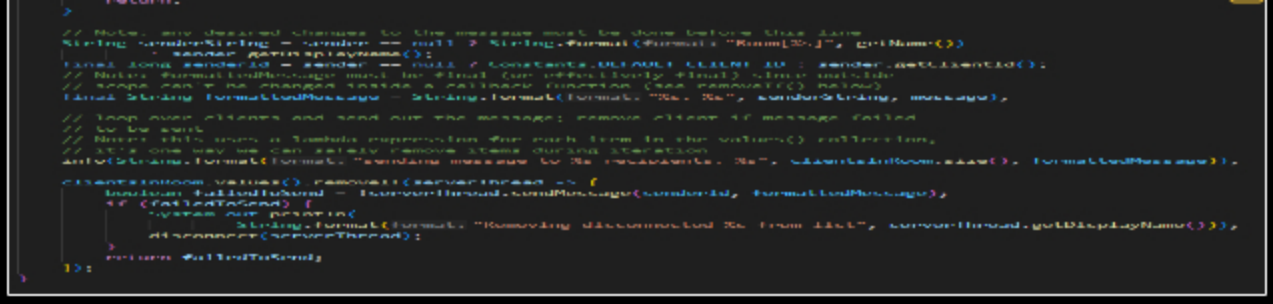
```java
//Rc728 11/5/25
private synchronized void relayToAllRooms(ServerThread sender, String message) {
    // Note: any desired changes to the message must be done before this line
    String senderString = sender == null ? "Server" : sender.getDisplayName();
    // Note: formattedMessage must be final (or effectively final) since outside
    // scope can't changed inside a callback function (see removeIf() below)
    final String formattedMessage = String.format("%s: %s", senderString, message);
    // end temp identifier

    // loop over Rooms and send out the message
    // Note: this uses a lambda expression for each item in the values() collection

    rooms.values().forEach(room -> {
        room.relay(sender, formattedMessage);
    });
}
```

this shows the code for how the server broadcast the message

```java
//Rc728 11/5/25
protected synchronized void handleMessage(ServerThread sender, String text) {
    relay(sender, text);
}
// end handle methods
```

This shows the code for the relay method to be called





this shows the code that allows for the message to be relayed to everyone in the room

Saved: 11/5/2025 9:33:08 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the message code flow works

Your Response:

When the client sends a message, a payload with the type message is sent to the sever which the server receives it in serverthread. It detects the message type and passes the message to the client's room. The room will run the relay method then sends the message to all the clients in the same room.

Saved: 11/5/2025 9:33:08 PM

Section #7: (1 pt.) Feature: Disconnection

Progress: 100%

Task #1 (1 pt.) - Evidence

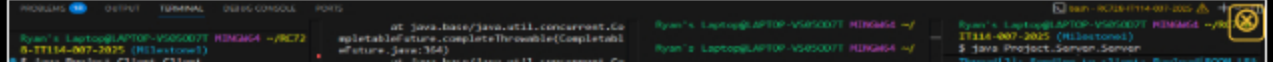
Progress: 100%

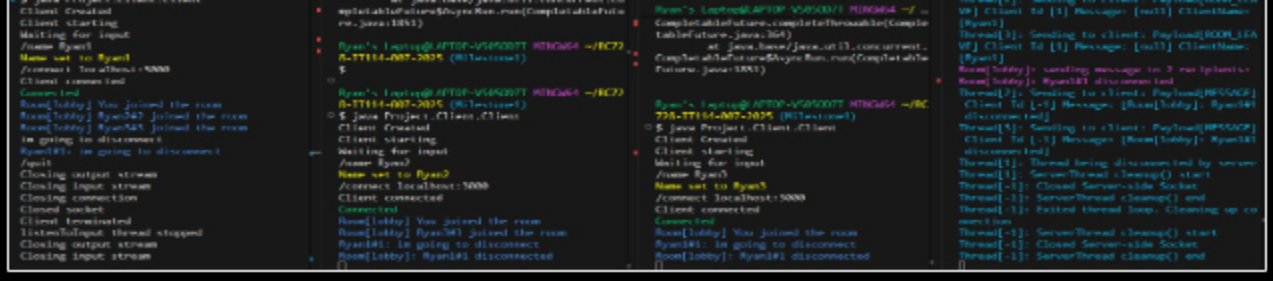
Part 1:

Progress: 100%

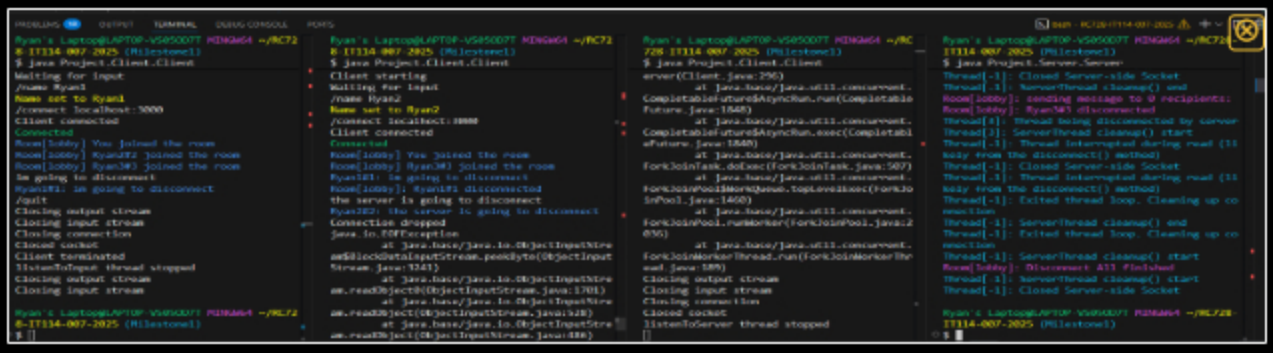
Details:

- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occuring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process

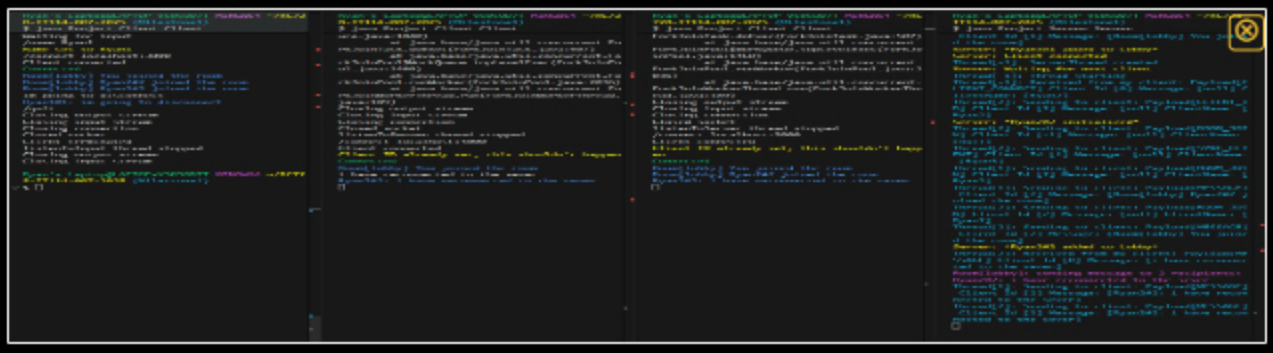




this shows the client disconnecting and the server still active



This shows the server disconnecting but the clients are still active



This shows the clients reconnecting to the server after it has reconnected

```java
    /**
     * Sends a disconnect action to the server
     *
     * @throws IOException
     */
    //rc728 11/5/25
    private void sendDisconnect() throws IOException {
        Payload payload = new Payload();
        payload.setPayloadType(PayloadType.DISCONNECT);
        sendToServer(payload);
    }
```

this shows the code for the client Side Disconnection

```java
    /**
     * Gracefully disconnect clients
     */
    //rc728 11/5/25
    private void shutdown() {
        try {
            // chose removeIf over forEach to avoid potential
            // ConcurrentModificationException
            // since empty rooms tell the server to remove themselves
            rooms.values().removeIf(room -> {
                room.disconnectAll();
                return true;
            });
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

this shows the code for the server side disconnect

Saved: 11/5/2025 11:04:55 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how both client and server gracefully handle their disconnect/termination logic

Your Response:

for the client disconnects it sends a payload object with the type disconnect which is sent over to the sever. This then ensures that the server knows the client is leaving and allows the sever to remove the client from the room cleanly. The server disconnects by using the shutdown() methods which goes through all the active rooms and uses the room.disconnectAll() method to safely disconnect all the clients in the room. This ensures that all the clients receives a disconnect message and the resources are cleaned up before the server is shutdown.

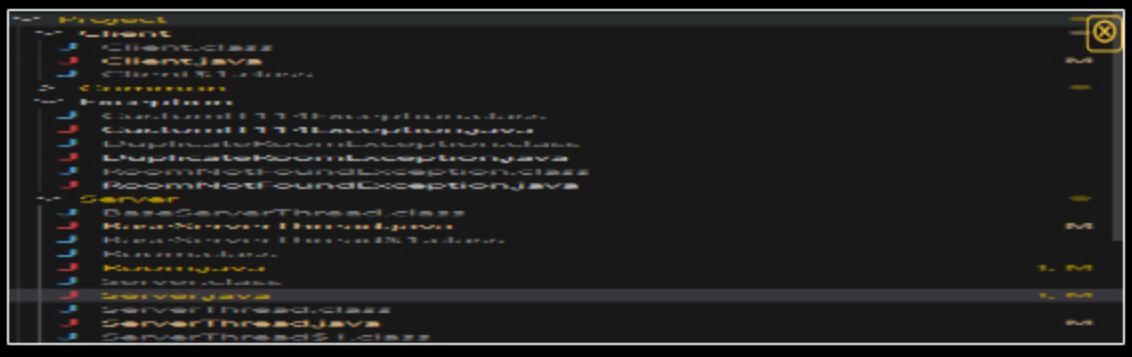
Saved: 11/5/2025 11:04:55 PM

Section #8: (1 pt.) Misc

Progress: 100%

Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages

Progress: 100%



this shows all the folders and files that is used in the project

Saved: 11/5/2025 11:06:55 PM

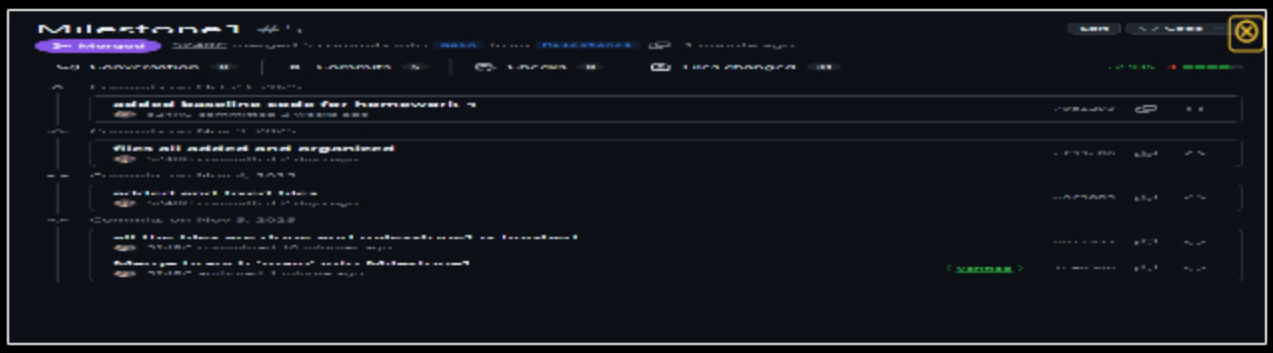
Task #2 (0.25 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



this shows all the commits

Saved: 11/5/2025 11:19:12 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/524RC/RC728-IT114-007-2025/

URL
https://github.com/524RC/RC728

Saved: 11/5/2025 11:19:12 PM

Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

this shows the top part of wakatime

this shows the bottom part of wakatime

Saved: 11/5/2025 11:30:12 PM

Task #4 (0.25 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I learned more about how the client and server works and all the different systems. I learned more about each file in the program when setting them up and got to know what each file does for the program.

Saved: 11/5/2025 11:38:36 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was using the program and putting the files into the folders. I was able to correctly put the files in the correct folders and got to learned more about each file and what they do for the project. I also was about to use the client and server without any issues after everything was fixed.

Saved: 11/5/2025 11:35:38 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was the setup of the files. I ran into a lot of issues setting up the files with changing certain text and troubleshooting to see if they work properly.

Saved: 11/5/2025 11:36:13 PM